Your first five minutes

Module 4: Talent Check

Explain how twelve stated skill levels compare with one company's expectations.

You start with an empty folder. Your Python backend handles logic; your React frontend is the browser screen. Use ordinary ChatGPT Free or Claude Free for help. You need no completed project, autonomous agent or teammate server. JSON means text with named data fields; it is how your parts exchange results.

A concrete example

For practice only, Synthetic Practice Co expects level 8 in every category. A fictional student enters level 4 in all twelve. Each ratio is 4/8=0.5, so the readiness score is 50%. Those are invented inputs, not real employer standards or verified ability.

Where your work fits

Profile Builder supplies a saved profile. A separate authorized source supplies company expectations. You return readiness and twelve gap rows. Skill Matching addresses a particular posting instead; its score is not this score.

The full journey is: JD Analytics -> Resume Parsing -> Profile Builder -> Talent Check -> Skill Matching. The last two are different comparisons: company-wide readiness and one specific job.

1. Load profile	**2. Choose source**	**3. Check 12 levels**
4. Compare rows	**5. Average ratios**	**6. Show gaps**

Do this first

- 1. Install the tools in the next section and create your own project folder.
- 2. Make the tiny health endpoint work. An API is how the browser asks your Python program for data; an endpoint is one address on that API.
- 3. Build one milestone at a time. You type/copy code, save files, run commands and inspect errors yourself. This PDF contains every required fictional input and agreed contract.

Read the requirements next, then start installing. Keep this PDF open beside VS Code. The PDF-only library is a reading resource; do not clone a finished app or upload your source into that library.

What counts as complete?

Organizer requirements

- [ ] Accept a saved profile and a selected company, then compare across all twelve official RADIX categories.
- [ ] Show a readiness score and per-category gaps using a documented numerical rule. The organizer permits a team scoring choice; the formula below is our design decision.
- [ ] Use genuine organizer expectations for official results. Current announced source: read-only staging_company_skill_levels in Supabase; actual access/schema are not supplied.
- [ ] Run independently with explicit synthetic fixtures now; show visibly different results for different candidates. Original-data acceptance and integrated end-to-end validation remain later gates.

ProofPath project requirements

- [ ] Show the arithmetic and each row's contribution, not only a circular percentage.
- [ ] Keep unknown competency null and overall score incomplete if any category is unknown.
- [ ] Label the source/mode and keep evidence confidence separate from self-reported competency.
- [ ] Write one pure scoring function that Career GPS can reuse for temporary category changes.

Optional later work

Radar charts, weighted categories, employer-specific cutoffs and live cloud deployment are optional decisions. Do not invent weights or official standards to make the score look impressive.

Be honest about missing resources

The original six JDs, four resumes and real company benchmarks have not been supplied. All examples here are invented. Build and test with them now; mark official-pack and authorized cloud acceptance pending. Synthetic success is not official-data acceptance.

An LLM is a language model that interprets text; Supabase is a cloud database service. The brief's JSON was a starting point. These manuals agree a simpler TEAM-V1 contract, different from the completed app's 1.0 format. The Scrum Master must build the explicit adapter described near the end; do not claim they already plug together.

Install your tools and open a folder

- 1. Install Python 3.12 from python.org/downloads/windows. Enable its PATH option. Python runs the backend: the program that reads files, validates data and calculates results.
- 2. Install Node.js 24 LTS from nodejs.org/en/download. It includes npm, a tool that installs JavaScript libraries. React runs the frontend: the screen in the browser.
- 3. Install VS Code from code.visualstudio.com/download and Git for Windows from git-scm.com/downloads/win. Git records versions of your work; GitHub stores a copy online.
- 4. Reopen PowerShell. Run the version checks below. If a command is not found, restart the terminal and check installation/PATH before continuing.

```
python --version
node --version
npm.cmd --version
git --version
```

- 5. Choose a location such as Documents. In PowerShell create your own project; code . opens it in VS Code. If code is unavailable, use VS Code -> File -> Open Folder.

```
mkdir radix-talent-check
cd radix-talent-check
code .
mkdir backend
mkdir samples
mkdir backend/tests
python -m venv .venv
```

In VS Code use Terminal -> New Terminal and choose PowerShell. Every command below says whether it starts at the project root or frontend/. The root is the folder containing backend/ and frontend/. .venv is an isolated Python dependency folder, not a file you write. Commands use its full Python path so you do not need to change PowerShell execution policy.

In the Explorer panel, right-click a folder -> New File. Create backend/__init__.py as an empty file. Names matter: app.py is a Python file; App.jsx is a React file. Save with Ctrl+S.

Make your first working backend

From the project root install the libraries. FastAPI creates API routes; Uvicorn runs the server; Pydantic checks data types; pytest runs tests; httpx sends HTTP requests and supports local API tests. python-multipart handles uploaded forms.

A backtick (`) at the end of a command continues it on the next line in PowerShell. Copy the whole command, including its continuation lines; leave no spaces after the backtick.

```
.\.venv\Scripts\python.exe -m pip install fastapi "uvicorn[standard]" pydantic `
  python-multipart httpx pytest python-dotenv pypdf python-docx
```

pypdf reads PDF text; python-docx reads Word files. Modules 4/5 mostly use JSON, but the shared install list is harmless. This is installation, not your finished module. Create backend/app.py and type this small health check:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://127.0.0.1:5173"],
    allow_methods=["GET", "POST", "PATCH", "DELETE"],
    allow_headers=["Content-Type"],
)

@app.get("/api/team/v1/health")
def health():
    return {"status": "ok"}

.\.venv\Scripts\python.exe -m uvicorn backend.app:app --reload --host 127.0.0.1 `
  --port 8000
```

Keep this terminal running. Open http://127.0.0.1:8000/api/team/v1/health and expect {"status":"ok"}. Also open http://127.0.0.1:8000/docs. Swagger /docs is an automatically generated form for calling your endpoints; choose Try it out, enter fields, then Execute.

CORS is the browser's cross-origin permission check. Your screen on port 5173 needs permission to call port 8000. Use the exact 127.0.0.1 addresses throughout; localhost is a different origin. This is local single-user development, not a production authentication design.

Start React and use web AI properly

Open a SECOND terminal at the project root; leave Python running in the first. Vite creates a small React project. Choose React/JavaScript if prompted; decline optional extra features for now.

```
npm.cmd create vite@latest frontend -- --template react
cd frontend
npm.cmd install
npm.cmd run dev -- --host 127.0.0.1 --port 5173 --strictPort
```

Open http://127.0.0.1:5173/. Seeing the Vite/React starter is your first frontend success. Later replace frontend/src/App.jsx and App.css. Never run the create command again over your work. If scripts are blocked, the commands above use npm.cmd directly without bypassing device policies.

Your browser tutor cannot see your computer

Open a normal ChatGPT or Claude web chat. Upload this PDF if your account permits it. Otherwise paste the purpose, current milestone and relevant contract pages. Free accounts may limit uploads or messages; work in small steps and keep local notes so a new chat can continue.

Ask for one or two named files at a time. Copy the returned code into the exact files yourself, save, run the command yourself, and paste the safe output back. Never say 'read my repo' unless you actually paste/upload its files. No tool subscription or IDE agent is assumed.

Copy this first prompt

I am building RADIX module 4, Talent Check, from scratch on Windows with basic programming knowledge. You are my web-chat tutor, not an agent on my computer. I will create files, copy code and run commands myself. Use this PDF's TEAM-V1 contract exactly. Work one milestone at a time, name each file, explain the code and wait for my actual result. Ask me to paste missing files or safe errors. Do not request keys, invent official data or assume an existing implementation.

At each milestone, also paste its Build requirements and the referenced full JSON. If AI proposes a different field or endpoint, compare it with this PDF before copying. Keep working versions in Git as you progress.

Understand the common data language

JSON is text made of named fields: strings use double quotes, lists use [], objects use {}, true/false are booleans and null means unknown/not present. Save JSON files as UTF-8. JSON does not allow comments or trailing commas. A schema is the agreed set of fields, types and rules.

When copying a PDF example, a viewer may insert line breaks inside a long quoted string. Join those visual wraps into one line, keeping literal \n escapes. Check your saved file with: python -m json.tool samples/profile.json (use your actual filename). A parse error must be fixed before testing the API.

Every object below is a complete fictional fixture, not a recording of live Groq output. IDs are short teaching examples; your backend should generate unique IDs. Compare meaning, references and arithmetic, not literal generated IDs or exact model confidence.

One category per skill

COD = Coding; DSA = Data Structures & Algorithms; OOD = Object-Oriented Design

APTI = Aptitude; COMM = Communication; AI = AI

CLOUD = Cloud; SQL = SQL; SWE = Software Engineering

SYSD = System Design; NETW = Networking; OS = Operating Systems

OTHER is allowed for a skill outside the twelve. It is not a thirteenth rating. Named technologies stay named: the category COD alone does not mean Python, and Java is not JavaScript.

Skill fields: all nine are present

- skill_id, skill_name and category_code: nonblank strings; IDs unique within a record. Category must be one of the codes above or OTHER.
- evidence: a short exact source quote for a document claim, or a clearly labelled manual reason. confidence: high, medium or low; this is NOT a 1-10 competency rating.
- source_type: document, manual or hypothetical. source_id: a source record ID or null for manual/hypothetical. source_page: positive integer if genuinely known, otherwise null (often Word).
- claim_status: claimed or needs_review. Keep negative/conflicting evidence for review. All claims remain unverified; never imply that review proves ability.

For document claims, the source ID must exist and the quote must match its source text (normalize whitespace only). Hypothetical claims are allowed only inside a temporary scenario, never in saved real profiles. Keep original source files private unless fictional and safe to share.

The complete Profile input

Create samples/rated_profile.json with this entire object. Every 4 is a deliberately invented self-rating for testing. It is not extracted from the fictional CV or assigned by an LLM. No parser or Profile Builder server is needed to use this input.


```
{
  "schema_version": "TEAM-V1",
  "id": "candidate-1",
  "revision": 1,
  "name": "Asha Example (fictional)",
  "email": "asha@example.test",
  "education": ["Fictional College - BSc Computing"],
  "skills": [
    {"skill_id": "cv-python", "skill_name": "Python", "category_code": "COD",
  "evidence": "Skills: Python", "confidence": "medium", "source_type": "document",
  "source_id": "resume-1", "source_page": null, "claim_status": "claimed"}
  ],
  "hackathons": [],
  "internships": [],
  "certifications": [],
  "preferred_roles": ["Junior Developer"],
  "projects": ["Built a Python quiz."],
  "experience": [],
  "achievements": [],
  "cv_file": "resume-1",
  "ratings": {"COD": 4, "DSA": 4, "OOD": 4, "APTI": 4, "COMM": 4, "AI": 4, "CLOUD": 4,
   "SQL": 4, "SWE": 4, "SYSD": 4, "NETW": 4, "OS": 4},
  "documents": [
    {"source_id": "resume-1", "source_type": "resume", "source_file": "cv_a.docx",
  "source_text": "Name: Asha Example (fictional)\nEmail: asha@example.test\nSkills:
  Python\nProjects: Built a Python quiz.\nEducation: Fictional College - BSc
  Computing"}
  ]
}
```

The complete synthetic Company input

Create samples/company.json from this object. It is a teaching file you create yourself; it is NOT an organizer export or a real employer bar. Require exactly these twelve level keys with finite values 1-10, and reject booleans. Never offer this record as official data.

```
{
  "schema_version": "TEAM-V1",
  "id": "synthetic-company-1",
  "company": "Synthetic Practice Co",
  "source_mode": "synthetic",
  "notice": "Invented teaching values. Not official company expectations.",
  "levels": {"COD": 8, "DSA": 8, "OOD": 8, "APTI": 8, "COMM": 8, "AI": 8, "CLOUD": 8,
  "SQL": 8, "SWE": 8, "SYSD": 8, "NETW": 8, "OS": 8}
}
```

Our formula, worked by hand

For COD: candidate 4 / required 8 = 0.5 attainment. Its share is 100 x 0.5 / 12 = 4.1667 displayed points. All twelve equal ratios give 50.0 overall. Changing only COD to 8 gives (1 + 11 x 0.5) / 12 x 100 = 54.2 after final rounding.

A missing rating gives null for that row's gap, shortfall, attainment and contribution; overall readiness_score=null, status=incomplete and rating_completeness=11. Do not fill unknown with zero.

Complete readiness response

This is the full expected TEAM-V1 result for the previous two input files in synthetic mode. Save it as samples/expected_readiness.json if helpful. Compute the score from unrounded ratios, not the rounded displayed contributions.


```
{
  "schema_version": "TEAM-V1",
  "company": "Synthetic Practice Co",
  "company_id": "synthetic-company-1",
  "profile_id": "candidate-1",
  "profile_revision": 1,
  "source_mode": "synthetic",
  "notice": "Invented teaching benchmark; self-reported levels, not a hiring
  probability.",
  "status": "complete",
  "rating_completeness": 12,
  "readiness_score": 50.0,
  "skillset_gap": [
    {"category_code": "COD", "required_level": 8, "candidate_level": 4, "gap": true,
  "shortfall": 4, "attainment": 0.5, "contribution": 4.1667},
    {"category_code": "DSA", "required_level": 8, "candidate_level": 4, "gap": true,
  "shortfall": 4, "attainment": 0.5, "contribution": 4.1667},
    {"category_code": "OOD", "required_level": 8, "candidate_level": 4, "gap": true,
  "shortfall": 4, "attainment": 0.5, "contribution": 4.1667},
    {"category_code": "APTI", "required_level": 8, "candidate_level": 4, "gap": true,
  "shortfall": 4, "attainment": 0.5, "contribution": 4.1667},
    {"category_code": "COMM", "required_level": 8, "candidate_level": 4, "gap": true,
  "shortfall": 4, "attainment": 0.5, "contribution": 4.1667},
    {"category_code": "AI", "required_level": 8, "candidate_level": 4, "gap": true,
  "shortfall": 4, "attainment": 0.5, "contribution": 4.1667},
    {"category_code": "CLOUD", "required_level": 8, "candidate_level": 4, "gap": true,
   "shortfall": 4, "attainment": 0.5, "contribution": 4.1667},
    {"category_code": "SQL", "required_level": 8, "candidate_level": 4, "gap": true,
  "shortfall": 4, "attainment": 0.5, "contribution": 4.1667},
    {"category_code": "SWE", "required_level": 8, "candidate_level": 4, "gap": true,
  "shortfall": 4, "attainment": 0.5, "contribution": 4.1667},
    {"category_code": "SYSD", "required_level": 8, "candidate_level": 4, "gap": true,
  "shortfall": 4, "attainment": 0.5, "contribution": 4.1667},
    {"category_code": "NETW", "required_level": 8, "candidate_level": 4, "gap": true,
  "shortfall": 4, "attainment": 0.5, "contribution": 4.1667},
    {"category_code": "OS", "required_level": 8, "candidate_level": 4, "gap": true,
  "shortfall": 4, "attainment": 0.5, "contribution": 4.1667}
  ]
}
```

Field types and validation for your results

All listed keys in the complete examples are required. Reject unknown fields rather than silently accepting a different version. schema_version is the exact string TEAM-V1; IDs are nonblank strings, revisions integers >=1, and all numbers must be finite (true/false are not numbers). Empty arrays are allowed where shown.

Profile input, in plain language

id/revision identify the saved version. name is nonblank; email may be empty but must have an email shape if supplied. education, projects, experience, hackathons, internships, certifications, preferred_roles and achievements are lists of strings. skills is a list of the common nine-field Skill objects. ratings contains exactly the twelve category keys, each null or a number 1-10. documents holds source_id/source_type/source_file/source_text strings, and cv_file is a source_id or null. Check source links and quotes; reject hypothetical skills in persisted input.

Company and ReadinessResult

Company id/company are strings; source_mode is synthetic or real; notice is a string; levels is exactly twelve category keys with numeric values 1-10. In this tutorial uploaded Company files are accepted only in explicitly selected synthetic mode. Real data must come from an authorized server-side source, never a client label.

The result has company/company_id/profile_id strings and integer profile_revision. source_mode/notice identify provenance. status is complete or incomplete; rating_completeness is integer 0-12; readiness_score is a number 0-100 or null.

skillset_gap has exactly twelve unique rows. Each row: category_code (official code), required_level (number 1-10), candidate_level (number 1-10 or null), gap (boolean or null), shortfall (nonnegative number or null), attainment (number 0-1 or null), contribution (number 0-8.3334 after display rounding, or null). Unknown candidate values make all derived row fields null. Sum unrounded ratios for the overall score.

Your agreed API addresses

These are NEW TEAM-V1 teaching endpoints, not the existing completed application's endpoints. Use the exact prefix so they cannot silently collide. Base URL: http://127.0.0.1:8000. The /docs page will show the routes after you implement them.

- POST /api/team/v1/talent-check/evaluate - multipart: profile_file (Profile JSON), company_file (Company JSON for synthetic mode), benchmark_mode (synthetic or real, required). Return ReadinessResult.
- GET /api/team/v1/talent-check/benchmark-status - No body. Return {available:false, source_mode:'real', message:'Authorized organizer source is not configured'} until actual real-source access has been implemented and verified. This is not a hidden synthetic fallback.

Request and response rules

JSON body endpoints receive application/json. Upload endpoints receive multipart/form-data, a browser form that can carry file bytes. Field names are case-sensitive. Let FormData or curl set the upload boundary; do not manually force a JSON Content-Type on an upload.

A schema-valid response can still be semantically wrong. Check the source words, references and numbers. On errors return a safe message in the common shape below with an appropriate non-2xx status. 415=unsupported type; 413=too large; 422=bad fields/content; 404=not found; 409=stale revision; 502=invalid provider output; 503=provider/source/storage unavailable.

```
{
  "error": {"code": "INVALID_INPUT", "message": "Choose a readable PDF or DOCX file."}
}
```

Call your backend by hand

- 1. Start Uvicorn from the project root in one terminal. Open /docs and select the route you are testing. Click Try it out, choose the fictional files and Execute.
- 2. Inspect the response JSON. For file-pair routes, choose the complete JSON files you typed from this PDF. IDs in saved responses may differ from teaching IDs; use current returned IDs/versions.
- 3. You can also use curl.exe in a second PowerShell terminal at the root. It is written explicitly because curl can be a PowerShell alias. The long command below is ONE line; PDF wrapping is only for reading.

```
curl.exe -X POST http://127.0.0.1:8000/api/team/v1/talent-check/evaluate -F `
  "profile_file=@samples/rated_profile.json" -F `
  "company_file=@samples/company.json" -F "benchmark_mode=synthetic"
```

For Modules 1/2 this command deliberately permits sending the fictional input to Groq. Do not run it against personal files without authorization. For a missing key, show a clear unavailable/manual state and leave live LLM acceptance pending.

Useful checks after you add routes

```
.\.venv\Scripts\python.exe -m pytest backend/tests -q
# From frontend/:
npm.cmd run build
```

A test is a small program with an expected result. pytest reports which assertions fail. Ask web AI to write the tests described in the milestone, inspect the expected values yourself, then run them locally. Do not replace a failing expected value with whatever the buggy code returned.

First debugging question

Does /docs return the right data? If yes, inspect frontend request/state. If no, debug the backend first. Keep the smallest failing input and the exact error. Never infer a working implementation from a nice screenshot or an HTTP 200 alone.

No Supabase access yet? Keep building

You have NOT been given the real Supabase URL/key, exact table columns, company identifiers or an authorized export. The organizer announced staging_company_skill_levels, which must remain read-only. Its existence does not tell you its schema or expected values.

What to do now

- Implement and test synthetic mode using the full Company JSON in this PDF. Show an obvious synthetic banner and source_mode=synthetic. Implement real-source status as unavailable; never silently switch to a lab result.

What is needed later

- An organizer-authorized project URL, read permission/credential, actual table schema (company/category/level fields or wide columns), company IDs/filters and any pagination rules. An authorized complete export can also be used for local acceptance.
- Only after those details arrive, add backend/supabase_source.py. Keep credentials in a private root .env, read with python-dotenv. Do not put privileged keys in React. Use read-only GET/SELECT; never create, update or delete organizer records.
- Validate all twelve real values before scoring. Record source, timestamp and authorized provenance. A client-uploaded file labelled real is not proof of authorization. Missing/invalid values give a source error, never guessed expectations.

Supabase is the target hosted database in the organizer stack. Local synthetic files are a truthful development alternative while access is missing. Mark actual Supabase/RLS and real-company acceptance pending until verified; neither this manual nor an account alone proves that integration.

Official Supabase Data REST API guide

Build milestone 1: Validate both inputs

What and why

A perfect formula on missing or fake standards is still a misleading result.

Create or edit these files

backend/models.py, backend/benchmarks.py, backend/tests/test_inputs.py

What belongs in them

Define Profile, Company and result models from the contract. Require exactly twelve unique expected-level codes, each a finite number 1-10 and not a boolean. Profile ratings are twelve finite numbers 1-10 or null. Synthetic mode requires source_mode=synthetic and an obvious notice. Real mode must remain unavailable until it reads an authorized source; changing an uploaded JSON label to real does not make it official.

Copy this prompt into your browser chat

You cannot access my computer. I will manually create/edit files, run commands and paste safe results. Using the TEAM-V1 objects I paste next, help write input models and benchmark validation. No scoring yet. Reject missing/extra categories, booleans, bad ranges and untrusted real-mode uploads. Give tests and explain how I run them manually.

Run it yourself

```
.\.venv\Scripts\python.exe -m pytest backend/tests/test_inputs.py -q
```

How to know it worked

The provided invented company validates. Removing OS or using level 0/true fails. No real-mode request silently returns the sample values.

If it goes wrong

Print safe category names and validation errors, never private credentials. Compare exactly against the category list, not merely a count of twelve.

Build milestone 2: Write the arithmetic once

What and why

Deterministic ordinary code is easier to explain than asking an LLM for a score.

Create or edit these files

backend/scoring.py, backend/tests/test_scoring.py

What belongs in them

Implement evaluate(profile, company) as a function with no network, file writes or mutation. For each category: attainment=min(candidate/required,1); contribution=100*attainment/12; shortfall=max(required-candidate,0). Round displayed contribution to four decimals, but compute the final score from unrounded ratios and round it to one decimal. Unknown produces null gap/ratio/points and a null overall score.

Copy this prompt into your browser chat

You cannot access my computer. I will manually create/edit files, run commands and paste safe results. Help write scoring.py using this exact formula and the full expected 50% response. Do not use an LLM, evidence confidence or candidate names. Give tests for all-4 versus all-8, all-8 versus all-8, one unknown, capping above the bar and repeatability. I will run them myself.

Run it yourself

```
.\.venv\Scripts\python.exe -m pytest backend/tests/test_scoring.py -q
```

How to know it worked

All 4 vs 8 gives 50.0; all 8 vs 8 gives 100.0; one unknown gives incomplete/null, not a fake zero. Adding a quote alone leaves arithmetic unchanged.

If it goes wrong

Check percentage versus ratio units and rounding. Do not sum already rounded displayed contributions to compute the final score.

Build milestone 3: Expose a source-aware API

What and why

The frontend needs one dependable comparison request and a clear missing-source explanation.

Create or edit these files

backend/app.py, backend/tests/test_api.py

What belongs in them

Add the evaluate and benchmark-status endpoints. Parse JSON uploads, validate, run the same scorer and return exactly twelve rows with provenance. Use 422 for invalid input and 503 when the selected real source is unavailable. Preserve the caller's mode selection; never replace missing official data with a synthetic company.

Copy this prompt into your browser chat

You cannot access my computer. I will manually create/edit files, run commands and paste safe results. I will paste models.py, scoring.py and app.py. Add the exact multipart endpoints from this guide. Include /docs tests for synthetic success, unknown ratings, malformed file and real-source unavailable. Do not write any organizer table or create invented official values.

Run it yourself

```
.\.venv\Scripts\python.exe -m pytest backend/tests/test_api.py -q
```

How to know it worked

The 50% result returns twelve explainable rows and a synthetic notice. Real mode without access returns a source error rather than a percentage.

If it goes wrong

If /docs shows 422, compare multipart field names. If a real-source problem looks like 0% readiness, the error handling is wrong.

Build milestone 4: Build a comparison a student can read

What and why

Twelve numbers need labels and explanation to become useful feedback.

Create or edit these files

frontend/src/App.jsx, frontend/src/App.css

What belongs in them

Allow Profile JSON and synthetic Company JSON upload plus an explicit source mode. Add Run Talent Check, an overall score or Incomplete state and twelve rows with category label, required/candidate values, gap and contribution. Use paired bars on a 1-10 scale if useful; never hide Unknown as a zero-length ability bar. Show source notice and explain one row on click.

Copy this prompt into your browser chat

You cannot access my computer. I will manually create/edit files, run commands and paste safe results. Help build my React Talent Check page against this contract. I will paste endpoint examples. Use readable labels, a visible synthetic banner, all twelve rows, unknown/error states and simple CSS bars. Do not hardcode 50% or invent a real company menu.

Run it yourself

```
cd frontend
npm run dev -- --host 127.0.0.1 --port 5173 --strictPort
```

How to know it worked

Upload fixtures, get 50%, inspect COD's 4.1667 points, change an input and get changed mathematics. Resize the browser: every row remains readable.

If it goes wrong

Inspect the API JSON before blaming the chart. Ensure required and candidate values share the same scale and category order.

Test normal cases before polishing

Create the tests yourself with web-chat help. Before asking AI for expected values, read the fictional inputs and calculate the expected meaning/math below. These are requirements for your implementation, not tests already run on code supplied with this PDF.

- Normal test 1: All ratings 4; all expectations 8. Expected: 50.0 readiness, twelve gaps, shortfall 4 in each row and contribution 4.1667 each.
- Normal test 2: All ratings 8; expectations 8. Expected: 100.0, no gaps and full capped attainment. A different candidate changes the result.
- Normal test 3: Only COD changes from 4 to 8. Expected: 54.2 readiness. Other rows stay the same; adding an evidence quote with ratings unchanged leaves 50.0.

Check the actual browser

Run Python and React in separate terminals. Walk through one complete successful interaction and inspect the downloaded/returned JSON. Resize to about 390 px wide: labels, action buttons, quotes and comparison rows must remain readable without horizontal clipping. Use keyboard Tab and meaningful labels.

Use body text at least 16 px, a comfortable line height, clear headings, a visible primary action, loading feedback, a helpful empty state and an error that preserves useful input. Simple navy/blue on white and ordinary CSS are enough; do not spend the build phase on decorative dashboards.

Prove the answer is not hardcoded

Change a required skill or candidate rating, rename the fictional person/company without changing skills, reorder input fields and use a genuinely different document. Relevant inputs must change results; irrelevant names must not. Repeat identical scoring inputs and expect identical arithmetic. Language-model wording/confidence can vary; deterministic scoring must not.

Test failures and unfamiliar inputs

- Failure test 1: One unknown rating. Expected: Twelve rows remain, unknown row fields are null, status=incomplete, completeness=11 and overall score=null.
- Failure test 2: Invalid benchmarks. Expected: Remove a category, add OTHER, duplicate a code in a converted array, or use 0/11/true. Reject; do not fill guesses.
- Failure test 3: Missing official source / network error. Expected: Real mode is unavailable with an actionable error. No synthetic standards are shown as official; cached data, if later added, must show its source/time and staleness.

Try an unfamiliar authorized document

Use only material you own or are permitted to process. For a new JD/CV, write a small expected list before extraction: skill, category, supporting quote, section and required/preferred/alternative meaning. Count unsupported extra claims (false positives) and missed supported claims (false negatives). Inspect names/background separately. More extracted skills is not automatically better.

For Modules 3-5, make a new fictional/authorized profile or reviewed JD JSON from those facts and test the same save/compare functions. Keep ground-truth notes separate from your parser output. A successful provider call or valid JSON proves neither truth nor competency.

Ask web AI to help debug one failure

I ran this command: [paste]. I expected: [specific result]. I got: [safe output or traceback]. Here is the smallest fictional input and the full relevant file [filename]. Diagnose this one failure, explain the cause and give a small correction plus a regression test. You cannot access my computer or run the fix. Do not change the contract, weaken the assertion or ask for my .env/key.

Paste a complete error from its first useful line, not only 'it doesn't work'. If API keys or personal text appear, remove them before sharing. After a fix, rerun the failing test and the other tests; keep the observed result, including failures.

Fix common setup and API mistakes

- python/node/git not recognized: confirm installation, PATH and a newly opened terminal. Use VS Code File -> Open Folder if code . is unavailable.
- ModuleNotFoundError: use .venv's Python and run from the root containing backend/. Confirm __init__.py exists. Installing into a different Python does not fix your environment.
- Address already in use: stop your own earlier server with Ctrl+C. Do not kill an unknown process. If you choose a different port, update frontend base URL and allowed CORS origin consistently.
- Cannot reach API: open the health URL and /docs first. Frontend requests should use http://127.0.0.1:8000, not the Vite port. A server traceback is more useful than a generic browser error.
- 422 response: inspect the response body and compare exact field names/types. Files use FormData; JSON bodies use application/json. Do not send a filename string when the API expects file bytes.
- JSON parse error: use straight double quotes, no comments/trailing commas and proper escaped \n inside strings. Copy the full object, not only the visible first page.
- CORS error: compare exact protocol/host/port. localhost and 127.0.0.1 differ. Add only your development frontend origin; do not use a wildcard as a substitute for understanding the failure.
- AI gives too much code: ask for one named file and one test. If a chat limit is reached, save your work and start a new chat with this PDF, the current milestone, relevant files and safe output.

A real working module has runnable source, correct output for changed inputs, tests that can fail and a UI that calls the backend. A screenshot or prerecorded JSON alone does not meet that standard.

Put your own code on your GitHub

The PDF-only repository is the library your Scrum Master shares. Your working code belongs in a DIFFERENT repository on YOUR GitHub account. Do not push code into the guide library, overwrite someone else's remote or send a screenshot as the only deliverable.

- 1. In your project root create .gitignore with the following lines. Keep real uploads in data/private/. Check the actual staged list before committing; ignore rules cannot erase a secret already committed.

```
.venv/
.env
.env.*
!.env.example
frontend/node_modules/
frontend/dist/
__pycache__/
.pytest_cache/
data/private/
*.log
```

- 2. In GitHub's normal website choose New repository, YOUR account, a clear name and Private. Leave automatic README, license and .gitignore unchecked because you already have local work. Add your own README.md locally: prerequisites, setup/run/test commands, contract version, examples and honest limitations.

- 3. From the project root run these commands. Replace YOU/YOUR-REPO with your actual new repository URL. Git may ask for your author name/email and open browser authentication. Do not paste a key/token into an AI chat.

```
git init -b main
git add .
git status --short
git diff --cached
git commit -m "Build and test my RADIX module"
git remote add origin https://github.com/YOU/YOUR-REPO.git
git push -u origin main
```

Before the commit, inspect for keys, .env, private CVs, source text from real people, databases and large dependency folders. Commit only source, lock/requirement files, tests, fictional fixtures and safe screenshots. Never force-push. If you already initialized Git during development, skip the init and use your existing branch intentionally.

Verify a fresh checkout and send the result

First freeze the Python dependencies that actually worked. From the ROOT run the first command below. Commit backend/requirements.txt and frontend/package-lock.json; do not copy .venv or node_modules. In your README state the Python/Node versions you used.

```
.\.venv\Scripts\python.exe -m pip freeze > backend/requirements.txt
git add backend/requirements.txt frontend/package-lock.json
git commit -m "Record verified dependencies"
git push
```

Now open a new terminal in another parent directory. Clone YOUR code repository into a new folder, not on top of your working folder. This catches missing files and accidental dependence on private paths.

```
git clone https://github.com/YOU/YOUR-REPO.git fresh-check
cd fresh-check
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r backend/requirements.txt
.\.venv\Scripts\python.exe -m pytest backend/tests -q
cd frontend
npm.cmd ci
npm.cmd run build
```

Start Python from the fresh root using the earlier Uvicorn command and React from fresh frontend/. Repeat one normal and one failure interaction. For Modules 1/2 add your private .env locally only if doing an authorized live test; offline/mock tests must not require a key. Do not commit that file.

Send the Scrum Master

- Your repository URL, module 4 name, and the output of git rev-parse HEAD. Invite the Scrum Master as a collaborator if the repository is private.
- Actual commands/results from the fresh checkout; three normal and three failure cases with expected/actual outcomes; input/output and error-recovery screenshots using fictional data.
- Complete TEAM-V1 output JSON and its fictional PDF/DOCX/JSON inputs in your own samples/ directory. Include changed-file notes and what is implemented, mock-tested, live-verified or pending.

You do not have to merge into the existing RADIX app or solve central conflicts. The Scrum Master evaluates your tested commit and builds the explicit bridge. Never claim your part is integrated until that separate verification succeeds.

Explain your work to judges

A short explanation to practise

I compare twelve explicit candidate self-ratings with a selected company's expected levels. The calculation is ordinary code, so I can explain every row and repeat it exactly. Unknown stays unknown. Our sample benchmark is clearly invented; official readiness needs authorized organizer data. Evidence links are useful context but cannot raise a rating. The scorer is a reusable function so Career GPS can preview changes consistently without altering a saved profile.

Your live demo sequence

Upload the fictional all-4 profile and all-8 benchmark; show 50%, all twelve rows and one worked contribution. Change COD to 8 for 54.2%. Clear one rating for Incomplete. Select real mode and show the honest unavailable message. Explain why the lab values are not employer standards.

Five likely questions

- Is 50% a hiring probability? No. It is the mean capped attainment for these self-ratings and expectations.
- Why cap each ratio at one? Exceeding one category's bar should not cancel a shortfall in another in this team formula.
- Why not score with an LLM? The comparison is transparent arithmetic; an LLM adds uncertainty without helping this calculation.
- What if a value is unknown? Show the row as unknown and withhold the overall score instead of inventing ability.
- Are the sample company levels official? No. They are invented, labelled and used only for development. Real acceptance is pending authorized data.

What makes it more than an AI wrapper?

The model, where used, is only one step. Validation, review, evidence traceability, persistence, transparent arithmetic, meaningful failures and non-mutating scenarios are ordinary software responsibilities that you can demonstrate and test. Show those behaviors, not just the model's paragraph.

Integration: an explicit adapter is required

This page is also for your Scrum Master. TEAM-V1 is a deliberate smaller teaching format. Your independent endpoints live under /api/team/v1/. The completed app uses /api/jd, /api/resumes, /api/profiles, /api/talent-check and /api/matching with its richer schema_version=1.0 models. Sending TEAM-V1 directly to those routes will fail validation; do not silently rename only the URL.

The bridge is NOT implemented by this PDF task. Students return working independent code and full fictional exports. The Scrum Master imports/adapts the data, registers authorized originals in the existing owner's private document store, maps IDs, then reruns existing canonical comparisons. Do not paste incoming JSON straight into the production store or mark it verified.

- Common Skill mapping: retain name/category/evidence/confidence. Map TEAM-V1 source_id to a newly registered owned source_document_id. Preserve source_type/claim_status. Add current skill_family, evidence_type, verification_status=unverified and other safe defaults. Derive/validate spans against actual source text; do not invent offsets or ownership.
- Benchmark mapping: convert Company.levels only from an explicitly synthetic fixture or a genuinely authorized real source. A source_mode string alone does not prove trust. The canonical benchmark repository must validate all twelve values and attach its actual source/hash/timestamp.
- Result mapping: TEAM-V1 omits the canonical row evidence, rating_origin, source snapshot and receipt metadata. Recompute using the canonical saved Profile and validated benchmark to produce ReadinessResult; never fabricate benchmark_hash or receipt_id to make a small result fit.

Integration acceptance: same intended input facts, preserved source quotes and unknown values, consistent expected arithmetic, all five canonical-module tests and one full candidate/JD/company journey. Actual real Groq/Supabase/organizer-data tests remain separately pending until resources exist. No deployment or existing project modification is part of distributing these manuals.

Finish honestly and keep these references

Final completion checklist

- [ ] My source runs from a fresh checkout with the documented commands and dependencies.
- [ ] All organizer inputs/outputs for my role work; all project extensions marked required are demonstrated.
- [ ] Three normal and three failure cases have actual results, not only expected results.
- [ ] I exported full TEAM-V1 JSON and included fictional source fixtures and screenshots in my own code repository.
- [ ] I state what is implemented, synthetic-tested, mock-tested, live-verified and pending. I do not claim that a key or HTTP 200 proves semantic quality.
- [ ] No private key, real CV/source text, private database or organizer export is committed. The Scrum Master can access my private repository and tested commit.

Known limits

The official pack and genuine company standards remain unavailable. OCR, perfect layout support, cloud RLS/security and a live LLM evaluation are not magically supplied by this PDF. The larger completed app is preserved separately; this tutorial uses a different contract and needs its documented adapter. Do not claim integration or official-data acceptance until the Scrum Master verifies it.

Official references checked 26 October 2026

Python virtual environments

FastAPI first steps and CORS

Vite React setup

GitHub: create a repository

Organizer reference: RADIX Talent Match Participant Brief, role pages 7-8, shared contract pages 9-10 and definition of done page 13, plus the user's later read-only table announcement. Requirements are summarized here; you do not need those separate files to follow the tutorial.